EXPERIMENT-26

26. RSA Public-Key Encryption – Private Key Leakage

AIM

To write a C program demonstrating RSA public-key encryption and explain whether generating new public and private keys using the same modulus is safe after Bob's private key is leaked.

PROCEDURE

1. Choose two prime numbers \(p\) and \(q\).
2. Calculate the modulus \(n=pq\) and Euler's totient \(\phi(n)=(p-1)(q-1)\).
3. Choose a public exponent \(e\) and calculate the private exponent \(d\).
4. Encrypt the plaintext using \(C=M^e\bmod n\).
5. Decrypt the ciphertext using \(M=C^d\bmod n\).
6. Examine whether changing the keys while keeping the same modulus protects Bob's RSA system.

CODE (C)

```
#include <stdio.h>

long long modpow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;

    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp /= 2;
    }

    return result;
}

int main(void) {
    long long p = 61, q = 53;
```

```
    long long n = p * q;
    long long phi = (p - 1) * (q - 1);

    long long e = 17;
    long long d = 2753;
    long long message = 65;

    long long cipher = modpow(message, e, n);
    long long decrypted = modpow(cipher, d, n);

    printf("RSA modulus n = %lld\n", n);
    printf("Public key (e, n) = (%lld, %lld)\n", e, n);
    printf("Private key (d, n) = (%lld, %lld)\n", d, n);
    printf("Plaintext = %lld\n", message);
    printf("Encrypted ciphertext = %lld\n", cipher);
    printf("Decrypted plaintext = %lld\n", decrypted);

    printf("\nIf the private key leaks, changing e and d");
    printf(" while keeping n is NOT safe.\n");
    printf("Generate a new modulus with new prime numbers.\n");

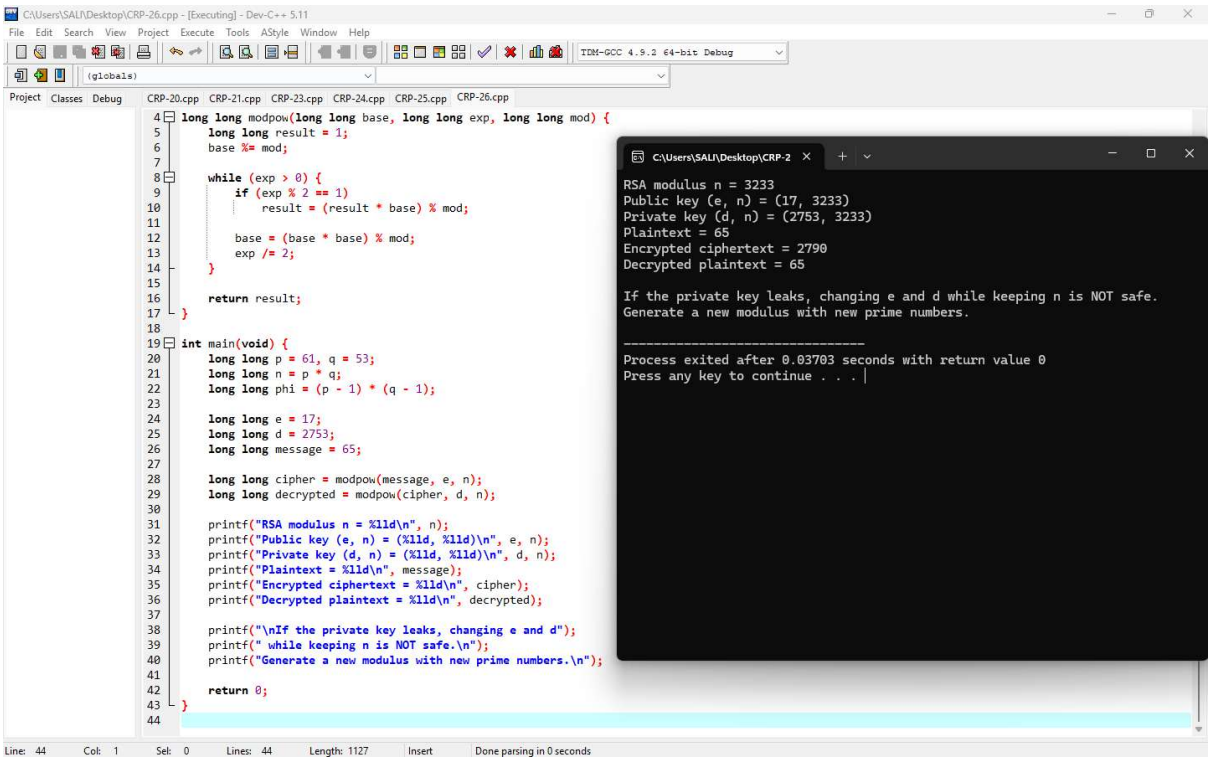
    return 0;
}
```

SAMPLE INPUT

```
Plaintext: 65
p = 61
q = 53
e = 17
d = 2753
```

SAMPLE OUTPUT

```
RSA modulus n = 3233
Public key (e, n) = (17, 3233)
Private key (d, n) = (2753, 3233)
Plaintext = 65
Encrypted ciphertext = 2790
Decrypted plaintext = 65
```



RESULT

The RSA program successfully encrypts and decrypts a message. The security analysis shows that Bob should generate a new modulus and new key pair if his private key is leaked.